

Cahier des Charges — Kiraa

Agent Intelligent de Location de Véhicules

ESISA – Fès, Maroc

Spécifications techniques et pédagogiques — Édition 100% Python

Table des matières

Introduction

Note Pédagogique

Ce cahier des charges définit la réalisation d'un agent intelligent de location de véhicules, dans une édition destinée aux étudiants souhaitant travailler exclusivement avec l'écosystème Python. Les étudiants doivent construire une application complète d'ingestion, d'extraction, d'orchestration, de calcul déterministe, de validation, de RAG, d'interface Web, de déploiement conteneurisé et de démonstration. La solution doit respecter exclusivement la pile technique Python décrite dans ce document – **aucune dépendance Node.js ou JavaScript n'est autorisée.**

Rôle des documents du projet

- Le présent cahier des charges → spécification de référence.
- `rental_policies.md` → base de connaissances textuelle utilisée exclusivement par le RAG.
- `CarRentalData.csv` → source brute importée par le script de seed Python vers PostgreSQL.

Le fichier de politiques ne doit pas être confondu avec les données opérationnelles. Les données de flotte, de clients et de réservations sont stockées dans PostgreSQL. Le RAG utilise exclusivement les passages pertinents de `rental_policies.md`.

Choix Techniques Obligatoires (100% Python)

- **API LLM** : utiliser une API LLM réelle, configurable et compatible avec l'extraction structurée, l'appel d'outils et les réponses en temps réel. Groq, OpenAI, Anthropic, Mistral ou un fournisseur équivalent sont acceptés via leur SDK Python officiel, si l'intégration est fonctionnelle et documentée.
- **Embeddings** : utiliser un fournisseur configurable (par exemple, SentenceTransformers en local, ou l'API d'embeddings du fournisseur LLM choisi). Les vecteurs doivent être compatibles avec PostgreSQL pgvector ; le modèle, la dimension et la stratégie de similarité doivent être documentés.
- **ORM** : utiliser **SQLAlchemy 2.0** (style déclaratif) avec **Alembic** pour les migrations, et **Pydantic v2** pour partager des contrats de données stricts entre la base, l'API interne et l'interface Streamlit.

1. Résumé Exécutif

Domaine	Location de véhicules
Problème	Risque d'hallucination du LLM, tarification complexe avec contraintes saisonnières et vérification de l'éligibilité du permis.
Solution	Architecture agentique LangGraph (Python) à 7 couches, moteur déterministe Python pur, PostgreSQL avec pgvector, interface Streamlit et exécution reproductible avec Docker Compose.
Objectifs pédagogiques	Utilisation de Python 3.11+, Pydantic v2, LangGraph, SQLAlchemy, PostgreSQL, RAG vectoriel, Streamlit, Docker Compose et pytest dans une architecture sans hallucination.

TABLE 1 – Synthèse exécutive

2. Contexte Métier & Utilisateur Cible

Une agence marocaine de location de véhicules reçoit des demandes par plusieurs canaux. L'agent doit vérifier la disponibilité du véhicule, l'éligibilité du conducteur, le coût total exact, puis produire une réservation et répondre aux questions de politique commerciale concernant l'annulation, le kilométrage, l'assurance et la franchise.

Personas utilisateurs :

- **Agent de comptoir** : traite rapidement les requêtes avec des calculs exacts.
- **Responsable d'agence** : gère les litiges, les demandes hors barème et les revues humaines.

3. Le Produit à Construire & Parcours Utilisateur

User stories :

- En tant qu’agent de comptoir, je veux vérifier instantanément l’éligibilité du permis afin d’éviter les locations non conformes.
- En tant que client, je veux recevoir une explication claire du coût de location et des politiques applicables.

Contrôles Zero-Trust

- Fichier hors sujet ou corrompu → rejet et demande d’un nouveau fichier.
- Champ absent → état CLARIFICATION_REQUIRED.
- Date ambiguë → demande de clarification.
- Conflit entre formulaire, document et message → confirmation humaine.
- Extraction sous le seuil de confiance → revue humaine.

Scénarios de Test (Parcours E2E)

Scénario	Ce qui se passe	Ce que le système doit faire
Conducteur mineur	20 ans, permis depuis 1,5 an	Rejet automatique, motif explicite, aucun calcul de prix.
Permis expiré	Expiration dépassée de 3 mois	Blocage immédiat, même si l’âge est conforme.
Jeune conducteur	22 ans, permis depuis 2,5 ans, véhicule Premium	Éligible, caution majorée de 50% et escalade humaine.
Remise hors barème	Demande de réduction de 25%	Le Validator refuse le taux non autorisé et plafonne la remise à 15%.
Question de politique	Annulation 24 heures avant la prise en charge	Réponse issue uniquement du RAG, sans calcul financier.

Chaque scénario E2E doit appeler le graphe LangGraph (Python) et vérifier l’état partagé persistant. Les assertions doivent vérifier `booking_status`, `needs_human_review`, `escalation_reasons`, le montant de la caution, le plafond de remise, la présence des passages RAG et l’absence de calcul pour une question de politique.

Test du Client, du Professeur ou de l’Utilisateur

L’environnement doit être démarrable avec Docker Compose et fournir une interface **Streamlit** accessible dans un navigateur. Les procédures d’installation, de préparation de la base, de chargement initial des données, d’exécution des tests et d’accès à l’interface doivent être décrites dans le README, sans exiger de configuration manuelle de PostgreSQL.

L’interface Web doit permettre de télécharger une pièce d’identité ou un permis sous forme d’image ou de PDF, d’ajouter des fichiers PDF, JSON ou TXT, de saisir un message, de déclencher l’analyse, de consulter les résultats déterministes, l’explication, les sources RAG et le statut de revue humaine, puis de télécharger le rapport final.

Interface Web Streamlit

Streamlit fournit une interface légère, entièrement écrite en Python, accessible par navigateur (`streamlit run app.py`). Les modèles Pydantic v2 sont partagés entre l'interface, la couche métier et le graphe LangGraph, garantissant un contrat de données unique sans duplication de schéma. L'évaluation doit être possible dans l'environnement Docker Compose avec les procédures documentées. Aucun bundler JavaScript, aucun `package.json` et aucun `node_modules` ne doivent être présents dans le dépôt.

4. Architecture Agentique Attendue (7 Couches)

Couche	Agent	Responsabilités
1	Ingestor	Normalise les messages, images, PDF, JSON et TXT dans un nœud LangGraph. Utilise un pipeline OCR Python (pytesseract ou easyocr).
2	Extractor	Extrait les champs avec un modèle Pydantic v2 (<code>response_format</code> structuré ou <i>function calling</i>) et produit un état typé. Détecte l'intention.
3	Orchestrator	Implémente le StateGraph de LangGraph , route vers les nœuds, gère l'état partagé (TypedDict ou modèle Pydantic) et déclenche l'escalade humaine. Les checkpoints utilisent langgraph-checkpoint-postgres, l'adaptateur officiel PostgreSQL de LangGraph.
4	Calculator	Exécute les formules déterministes dans des fonctions Python pures (typées, sans effet de bord), sans appel LLM.
5	Validator	Applique les règles booléennes et retourne un résultat validé par Pydantic avec <code>is_valid</code> et la liste des erreurs.
6	Explainer	Explique les résultats en langage naturel sans modifier les montants, dates ou décisions.
7	Reporter	Génère le devis, le contrat PDF/DOCX (reportlab ou python-docx) ou le tableau d'analyse (pandas).

Le graphe doit reconnaître les intentions `check_availability`, `calculate_total_cost`, `validate_eligibility`, `make_reservation`, `policy_query`, `human_escalation` et `out_of_scope`.

Séparation des responsabilités

Le RAG est utilisé uniquement pour les questions de politique. Les données opérationnelles, la disponibilité, l'éligibilité et les calculs financiers sont traités par PostgreSQL et les fonctions Python déterministes. Le LLM ne calcule aucun montant et ne prend aucune décision réglementaire.

Axiome Zéro-Hallucination

Le code Python calcule et valide de manière déterministe. Le LLM orchestre, extrait et explique en langage naturel. Aucun calcul mathématique ni aucune décision réglementaire ne doivent être délégués à l'hallucination du LLM.

Seuils d'Escalade Humaine

- Confiance OCR de l'extraction < 0.85 .
- Jeune conducteur de 21 à 24 ans sur un véhicule Premium.
- Montant de caution requis $> 20\,000$ MAD.

5. Exigences du MVP

- EX-01 : Ingestion et normalisation** : ingestion des messages, images, PDF, JSON et TXT en Python pur (Pillow, pypdf/pdfplumber, pyesseract/easyocr).
- EX-02 : Extraction structurée** : modèles **Pydantic v2** et *type hints* Python pour toutes les données extraites.
- EX-03 : Moteur déterministe** : calculs Python purs pour le prix, la caution, l'éligibilité et les pénalités, sans appel LLM.
- EX-04 : Orchestration** : graphe d'état **LangGraph** avec les 7 couches et les 7 intentions.
- EX-05 : RAG de politique** : recherche vectorielle PostgreSQL avec l'extension **pgvector** via **psycopg** et le package Python **pgvector**.
- EX-06 : Explainer** : explication contrôlée des résultats sans modification des données.
- EX-07 : Interface Web** : application **Streamlit** multi-pages, accessible par navigateur.
- EX-08 : Base de données** : schémas PostgreSQL, migrations **Alembic**, tables opérationnelles et checkpoints **LangGraph**.
- EX-09 : Docker Compose** : orchestration conteneurisée reproductible de l'application Python, de PostgreSQL et de **pgvector**.
- EX-010 : Tests** : tests unitaires et E2E avec **pytest**.

6. Dataset, Moteurs Déterministes & Méthodologie

Les fichiers sources sont chargés dans PostgreSQL par un script de seed Python (SQLAlchemy + pandas) :

- `fleet_catalog.csv` → table `fleet_catalog`;
- `customer_profiles.csv` → table `customer_profiles`;
- `booking_logs.csv` → table `booking_logs`;
- `seasonal_pricing_matrix.csv` → table `seasonal_pricing_matrix`;
- `rental_policies.md` → corpus documentaire du RAG.

Logique Financière Python

$\text{Total} = (\text{Base_Price} \times \text{Days} \times \text{Seasonal_Mult}) + \text{Insurance} + \text{Deposit} - \text{Capped_Discount}$

- La remise est strictement plafonnée à 15%.
- La caution est augmentée de 50% pour les conducteurs de moins de 25 ans.

Les fonctions de calcul reçoivent des données validées par Pydantic et retournent des résultats typés (dataclasses ou modèles Pydantic). Elles ne doivent appeler ni LLM ni outil de génération de texte.

Méthodologie Recommandée en 5 Phases

1. Ingestion Python, OCR et validation des fichiers.
2. Extraction structurée avec une API LLM réelle et Pydantic v2.
3. Moteur déterministe Python et migrations PostgreSQL (Alembic).
4. LangGraph, RAG pgvector et checkpoints PostgreSQL.
5. Application Streamlit, Docker Compose et tests E2E pytest.

7. Rôle du Dataset et de la Base RAG

Le dataset constitue la mémoire opérationnelle structurée. Les tables PostgreSQL contiennent la flotte, les profils clients, les réservations et les tarifs saisonniers. **SQLAlchemy** et **Alembic** gèrent les modèles et migrations. Les questions de disponibilité et de prix sont traitées par PostgreSQL et Python.

Le fichier `rental_policies.md` est fragmenté et transformé en embeddings avec un fournisseur configurable (par exemple `sentence-transformers` en local, ou l'API d'embeddings du fournisseur LLM). Les fragments et leurs vecteurs sont stockés dans PostgreSQL avec une colonne `pgvector`; la recherche utilise la similarité cosinus. Le module RAG récupère les passages pertinents avant de générer une réponse fondée.

- disponibilité et prix → tables PostgreSQL et fonctions Python;
- politique d'annulation ou d'assurance → RAG avec `pgvector`;
- explication finale → Explainer LLM contrôlé.

8. Contrat d'Ingestion des Fichiers et Test Réel

L'application doit accepter les images JPG, JPEG et PNG, les documents PDF, les fichiers TXT et les données JSON. Les fichiers de test doivent comprendre au minimum une image, un PDF, un fichier JSON et un fichier TXT représentatifs.

L'image est traitée par un pipeline OCR Python (`pytesseract` ou `easyocr`). Le JSON est chargé et validé par un modèle **Pydantic**. Pour chaque fichier, le système affiche le nom, le type, le statut de validation, le moteur utilisé, le texte extrait ou le contenu structuré, le score de confiance, les erreurs et le statut de revue humaine.

Pour un PDF, le système tente d'abord l'extraction native du texte (`pypdf` ou `pdfplumber`). Si le texte exploitable est insuffisant, le PDF est converti en image (`pdf2image`) et envoyé au

pipeline OCR. Un document illisible, corrompu, hors sujet ou incomplet est rejeté ou placé dans l'état `CLARIFICATION_REQUIRED` ; aucun champ absent ne doit être inventé.

9. Livrables Attendus

Le dépôt doit comprendre :

- application **Streamlit** (`app.py` et pages associées) ;
- graphe **LangGraph** et nœuds Python ;
- fonctions métier déterministes (module Python pur, testable) ;
- modèles **SQLAlchemy** et migrations **Alembic** ;
- script de seed Python contrôlé et non destructif ;
- `Dockerfile`, configuration `docker-compose.yml` et fichiers d'exclusion associés (`.dockerignore`, `.gitignore`) ;
- fichier d'exemple de variables d'environnement (`.env.example`) sans valeurs secrètes ;
- tests **pytest** unitaires et E2E ;
- générateur PDF/DOCX (`reportlab/python-docx`) ;
- fichier `requirements.txt` ou `pyproject.toml` (Poetry/uv) figeant les versions ;
- README d'installation, d'exécution, de test et de déploiement distant ;
- documentation des ports, des URLs et des routes de healthcheck ;
- route ou endpoint applicatif de healthcheck ;
- procédure de rollback ou de retour à une version précédente ;
- rapport de readiness confirmant le fonctionnement de l'URL distante et des scénarios E2E ;
- démonstration fonctionnelle de 5 minutes.

Échéance et dépôt : la date limite, le nom du dépôt et le format de remise seront communiqués par l'encadrement. Chaque groupe doit utiliser la convention de nommage indiquée dans le README et fournir un dépôt reproductible contenant le code, la documentation et les tests.

9.1. Stratégie de Déploiement et Accès Client

Deux modes de déploiement sont obligatoires : le mode local d'évaluation et le mode distant de démonstration. Docker Compose constitue la référence pour la reproductibilité locale et l'évaluation. Si un hébergeur prend en charge Docker Compose, la configuration peut être utilisée ; sinon, ses services doivent être déployés comme composants équivalents et documentés. Dans les deux cas, PostgreSQL doit être persistant et compatible avec `pgvector`.

L'environnement local de test doit être reproductible avec Docker Compose (image Python officielle, par exemple `python:3.11-slim`). Le client peut lancer l'environnement documenté, appliquer les migrations Alembic et le seed contrôlé, puis ouvrir l'application Streamlit dans son navigateur. Pour la démonstration distante, l'équipe fournit une URL HTTPS publique protégée et déploie l'application conteneurisée ainsi qu'une base PostgreSQL persistante compatible avec `pgvector`. Un hébergement tel que Railway, Render, Fly.io ou un VPS Docker peut être utilisé à titre d'exemple, mais le choix doit être documenté et respecter les exigences de persistance, de sécurité, de santé des services, de gestion des secrets et de supervision.

9.2. Mode Local d'Évaluation

Le mode local doit fournir l'application Streamlit, PostgreSQL avec `pgvector`, les volumes persistants, un réseau interne Docker, des healthchecks PostgreSQL et application, ainsi qu'une initialisation idempotente. Les migrations Alembic, le seed, les tests unitaires et les tests E2E doivent être accessibles par des scripts documentés dans le README (par exemple via un `Makefile` ou des commandes `python -m`). Les variables d'environnement sont injectées sans secret écrit en dur. Le README précise si les migrations et le seed sont déclenchés automatiquement (`entrypoint.sh`) ou par des scripts vérifiables. Le seed ne doit jamais supprimer ou écraser involontairement les données d'un environnement distant.

9.3. Mode Distant de Démonstration

La procédure distante comprend la construction d'une image Docker Python versionnée, sa publication dans l'environnement d'hébergement, le provisionnement d'une base PostgreSQL persistante compatible avec `pgvector`, la configuration des variables d'environnement dans le gestionnaire de secrets, l'exécution des migrations Alembic avant la mise en service, l'exécution d'un seed contrôlé et non destructif, le démarrage de l'application Streamlit, la vérification du healthcheck HTTP et de la connexion PostgreSQL, l'activation de l'URL HTTPS publique et la remise au client des instructions d'accès et des limites connues.

Le déploiement distant exige une URL HTTPS, une base PostgreSQL persistante avec `pgvector`, des secrets protégés, des logs sans clés API, un healthcheck HTTP, un contrôle de santé PostgreSQL, un redémarrage automatique après échec, une limitation de la taille des fichiers téléversés, une protection des routes d'administration et une procédure d'arrêt, d'archivage, de suppression et de rollback. Railway, Render, Fly.io et un VPS Docker sont des exemples possibles, mais aucun fournisseur n'est obligatoire. Le README doit documenter le fournisseur choisi, ses limites, la persistance, le support `pgvector`, le mode Docker, le coût éventuel, le DNS, le HTTPS et l'URL publique.

9.4. Recette Client en Ligne

- L'URL HTTPS répond et l'interface Streamlit s'affiche dans un navigateur.
- Un prompt texte, une image, un PDF, un JSON et un TXT sont acceptés.
- L'ingestion affiche le statut, le contenu extrait ou structuré, la confiance, les erreurs et la revue humaine.
- Une question de politique utilise uniquement le RAG.
- Une question de disponibilité utilise PostgreSQL.
- Un calcul de prix respecte les fonctions déterministes.
- Le plafond de remise de 15% est respecté.
- La caution du jeune conducteur est majorée de 50%.
- Un scénario nécessitant une revue humaine reçoit le statut attendu.
- Les logs ne révèlent aucune clé API, aucun mot de passe ni aucune donnée sensible.
- L'application signale correctement une base indisponible ou une clé obligatoire manquante.
- Les données persistent après un redémarrage contrôlé du service.

Le README doit documenter les procédures locales et distantes, les variables d'environnement, les secrets, les ports, les URLs, la journalisation, le contrôle d'accès, la supervision et le rollback.

10. Bonus Valorisés

- **Génération PDF** : contrat de location automatisé avec les données validées (`reportlab`).
- **Interface vocale** : requêtes vocales en Darija (`speech_recognition` ou API vocale du fournisseur LLM).
- **Dashboard analytique** : page Streamlit supplémentaire (avec `plotly` ou `altair`) pour l'analyse de l'état de la flotte.

11. Éléments Communs & Grille d'Évaluation

Poids	Critère	Description
25%	Architecture agentique et LangGraph (Python)	Mise en œuvre des 7 couches, du graphe d'état, de la persistance de l'état, du routage des intentions, de l'escalade humaine et de l'axiome Zéro-Hallucination.
20%	Moteur déterministe et validation Pydantic	Exactitude des règles d'éligibilité, du calcul financier, du plafond de remise de 15%, de la majoration de caution de 50% et des contrats Pydantic.
15%	PostgreSQL et RAG vectoriel	Schémas relationnels SQLAlchemy, migrations Alembic, persistance, extension <code>pgvector</code> , ingestion de <code>rental_policies.md</code> et récupération des passages pertinents.
15%	Ingestion multi-format et OCR	Prise en charge des images, PDF, JSON et TXT, extraction native PDF, OCR de secours, validation, score de confiance et revue humaine.
15%	Tests pytest et qualité du code	Tests unitaires et E2E, scénarios critiques, gestion des erreurs, lisibilité, modularité, documentation et absence de secrets dans le dépôt.
10%	Interface Streamlit et démonstration distante	Interface Web accessible, parcours utilisateur clair, téléversement des fichiers, affichage des décisions, déploiement HTTPS et démonstration reproductible.

TABLE 2 – Grille d'évaluation

Stack Technique Commune (100% Python) : Python 3.11+, Pydantic v2, SQLAlchemy 2.0, Alembic, LangGraph, `langgraph-checkpoint-postgres`, PostgreSQL + `pgvector` (via `psycopg` et le package `pgvector`), API LLM réelle configurable (SDK Python), embeddings configurables, pytest, Streamlit, Docker & Docker Compose, et génération PDF/DOCX (`reportlab/python-docx`).

Pourquoi cette pile est recommandée pour les étudiants Python

- **Un seul langage** du backend à l’interface : plus besoin d’apprendre TypeScript, npm ou React pour livrer un produit complet.
- **LangGraph** est la bibliothèque native (Python) sur laquelle LangGraph.js a été portée ; elle bénéficie du support et de la documentation les plus complets.
- **Streamlit** permet de construire une interface de démonstration fonctionnelle en quelques dizaines de lignes, sans configuration de bundler.
- **SQLAlchemy + Alembic** est le standard de facto de l’écosystème Python pour l’ORM et les migrations, avec un support mature de **pgvector**.
- **pytest** offre un écosystème de tests plus riche (fixtures, paramétrage) que Vitest pour un projet Python.